Algorithm Engineering
Woche 1, Aufgabe 3

Jan Vogt

Coding Warmup (Folie 19)

Alle Programme habe ich auf meinem eigenen Laptop getestet (Apple M3 mit 4 Performance-Kernen und 4 Effizienz-Kernen). Auf dem Mac ist `g++` standardmäßig nur ein Verweis auf Clang, das `-fopenmp` nicht unterstützt. Deshalb habe ich den GCC von Homebrew benutzt, also zum Beispiel:

```
g++-16 -fopenmp hello.cpp -o hello
OMP_NUM_THREADS=4 ./hello
```

Die Dateien zu dieser Aufgabe liegen im selben Ordner: `hello.cpp` (Folie 12), `hello_omp.cpp` und `hello_thread.cpp` (Folie 14) sowie `pi_parallel.cpp` mit meiner parallelen Version des π-Programms.

1. Hello World Programme

Bei `hello.cpp` mit 4 Threads kam zum Beispiel folgende Ausgabe heraus:

```
Hello from thread Hello from thread Hello from thread Hello from thread 3
2
0
1
```

Die Ausgaben der Threads sind also durcheinander. Das liegt daran, dass die Zeile mit `cout` aus mehreren einzelnen Ausgaben (`<<`) besteht und die Threads gleichzeitig auf die Konsole schreiben. Die Reihenfolge der Threads ist außerdem bei jedem Start anders. Ohne `OMP_NUM_THREADS` startet OpenMP auf meinem Rechner 8 Threads, also einen pro Kern.

Die beiden Programme von Folie 14 geben beide viermal „Hello World!“ aus. Man sieht also, dass die OpenMP Version im Prinzip dasselbe macht wie die Version mit `std::thread`, nur dass man sich bei OpenMP nicht selbst um das Starten und Joinen der Threads kümmern muss.

2. Parallele Version des π-Programms

Die Musterlösung auf Folie 20 verteilt die Rechtecke reihum auf die Threads, also bekommt Thread 0 die Rechtecke 0, 4, 8 usw. Ich habe es etwas anders gemacht: Jeder Thread bekommt einen zusammenhängenden Block von Rechtecken. Bei 4 Threads rechnet Thread 0 also das erste Viertel, Thread 1 das zweite Viertel und so weiter.

```
int start = (long long)num_steps * id / anzahl;
int ende = (long long)num_steps * (id + 1) / anzahl;
```

Außerdem benutze ich kein `atomic`. Stattdessen gibt es einen `vector` mit einer Teilsumme pro Thread, in den jeder Thread am Ende einmal sein Ergebnis schreibt. Nach der parallelen Region werden die Teilsummen dann der Reihe nach addiert. Dadurch werden die Teilsummen immer in der gleichen Reihenfolge zusammengezählt und das Ergebnis ist bei gleicher Threadanzahl bei jedem Durchlauf exakt gleich.

Das Programm rechnet π nacheinander mit 1, 2, 4 und 8 Threads aus. Jede Messung wird 5 mal wiederholt und die beste Zeit genommen, weil einzelne Messungen ziemlich schwanken.

3. Zeiten mit verschiedenen Threadanzahlen

Mit 100.000.000 Rechtecken und ohne Optimierung kam Folgendes heraus:

Threads	π	Zeit [s]	Speedup
1	3.14159265359042639	0.1396	1.00
2	3.14159265358990991	0.0719	1.94
4	3.14159265358968254	0.0373	3.74
8	3.14159265358981488	0.0302	4.62

Bis 4 Threads wird das Programm fast linear schneller. Mit 8 Threads ist der Speedup aber nur noch etwa 4,6 und nicht 8. Das liegt vermutlich daran, dass der M3 nur 4 schnelle Kerne hat und die anderen 4 Effizienz-Kerne deutlich langsamer sind. Weil bei meiner Version jeder Thread gleich viele Rechtecke bekommt, müssen die Threads auf den schnellen Kernen am Ende auf die langsamen warten. Das ist mir auch im Zusammenhang mit Folie 17 aufgefallen: Die Rechner im PC-Pool haben ebenfalls P-Cores und E-Cores, dort müsste man also einen ähnlichen Effekt sehen.

Bei den Ergebnissen sieht man außerdem, dass sich die letzten Nachkommastellen von π je nach Threadanzahl leicht unterscheiden, obwohl alle auf etwa 12 Stellen genau sind. Das kommt daher, dass sich durch die Aufteilung die Reihenfolge der Additionen ändert (siehe auch Aufgabe 2).

4. Kompilieren mit -Ofast

Mit `g++-16 -fopenmp -Ofast pi_parallel.cpp` sahen die Zeiten so aus:

Threads	π	Zeit [s]	Speedup
1	3.14159265359002227	0.0331	1.00
2	3.14159265358973672	0.0169	1.96
4	3.14159265358970652	0.0087	3.82
8	3.14159265358974693	0.0070	4.73

Schon mit einem Thread ist das Programm mit `-Ofast` etwa 4 mal schneller als ohne Optimierung. Mit 8 Threads und `-Ofast` zusammen ist es ungefähr 20 mal schneller als die Version mit einem Thread ohne Optimierung. Der Speedup durch die Threads bleibt dabei ungefähr gleich.

Auffällig ist, dass sich die Werte für π auch zwischen den beiden Varianten unterscheiden. `-Ofast` schaltet unter anderem `-ffast-math` ein. Damit darf der Compiler Rechnungen mit Kommazahlen umstellen, auch wenn sich dadurch das Ergebnis minimal ändert. Für dieses Programm spielt das keine Rolle, weil die Abweichung von π in beiden Fällen bei ungefähr 10^{-13} liegt.